

**Реализация объектов:
наследование, инкапсуляция,
полиморфизм**



МИХАИЛ БОРОДИН

Backend Lead, AppFollow

X5 Tech,
FinEx

- Более 10 лет разработки на Python
- Автор и эксперт курсов по Python и Django

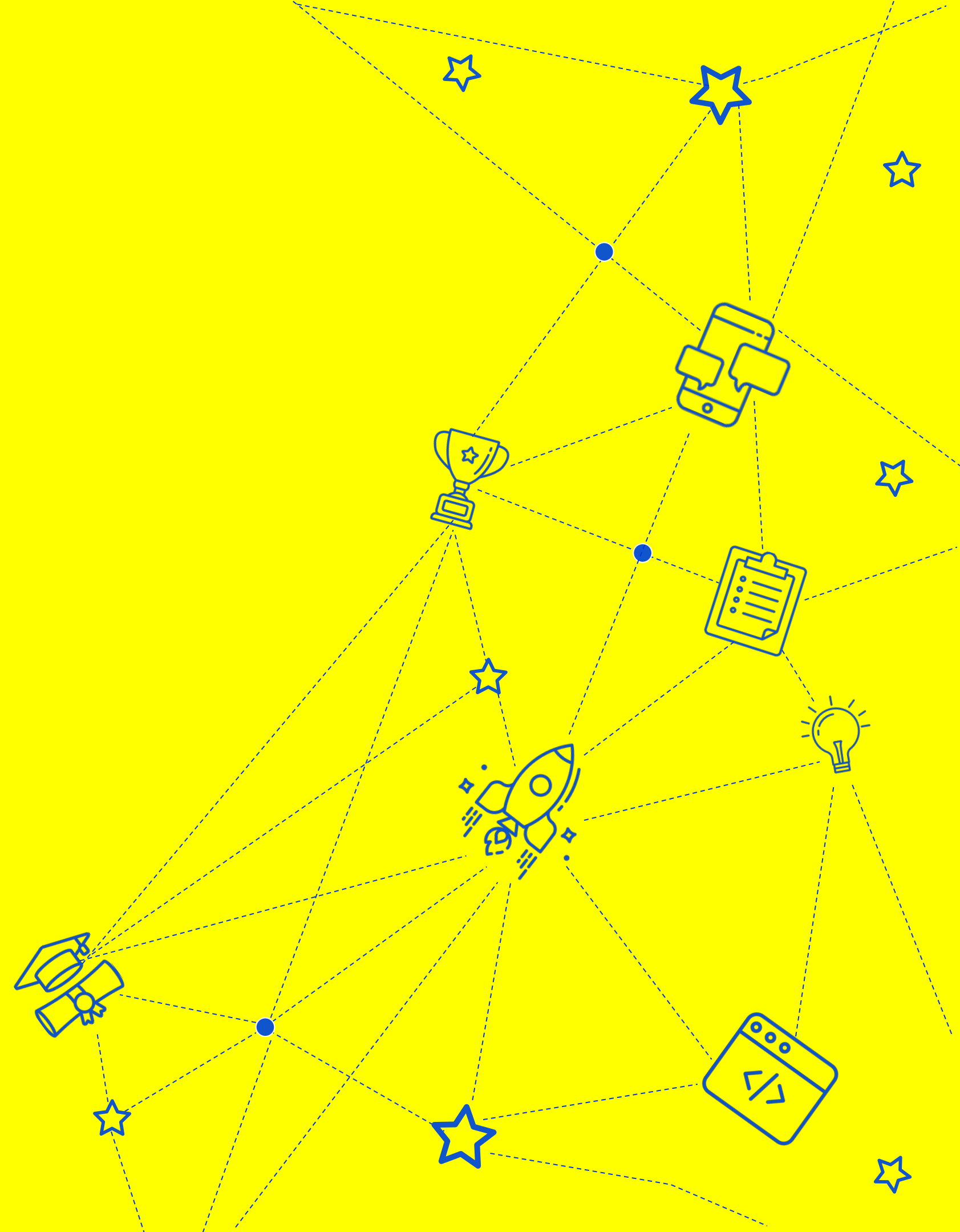
Содержание урока

☆ Наследование

☆ Инкапсуляция

☆ Полиморфизм

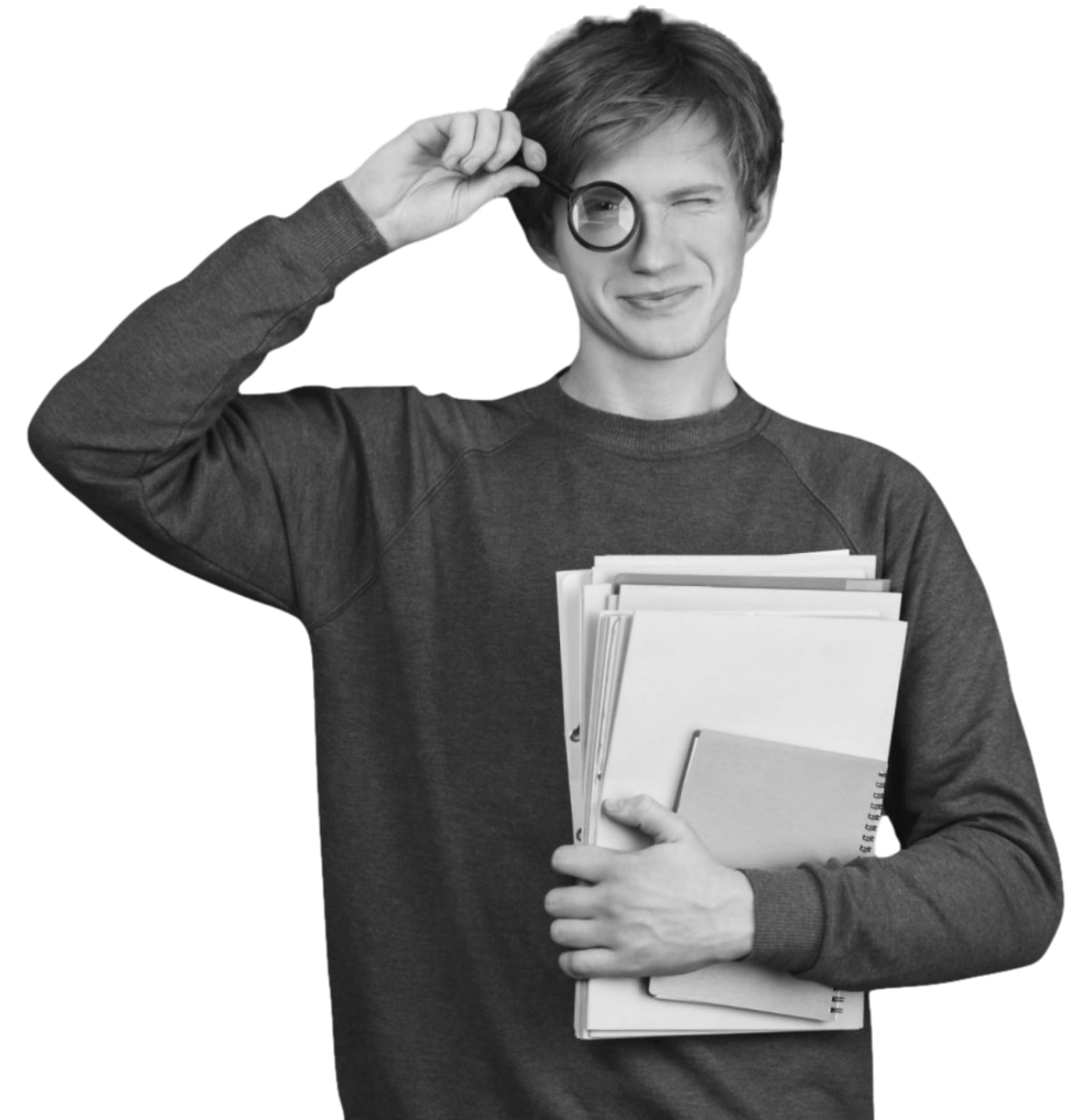
Наследование



Наследование

Что изучим

- 1 Что такое наследование
- 2 Суперклассы и подклассы
- 3 Как наследование реализовано в Python
- 4 Порядок наследования
- 5 Изменения порядка наследования
- 6 Абстрактные методы и классы

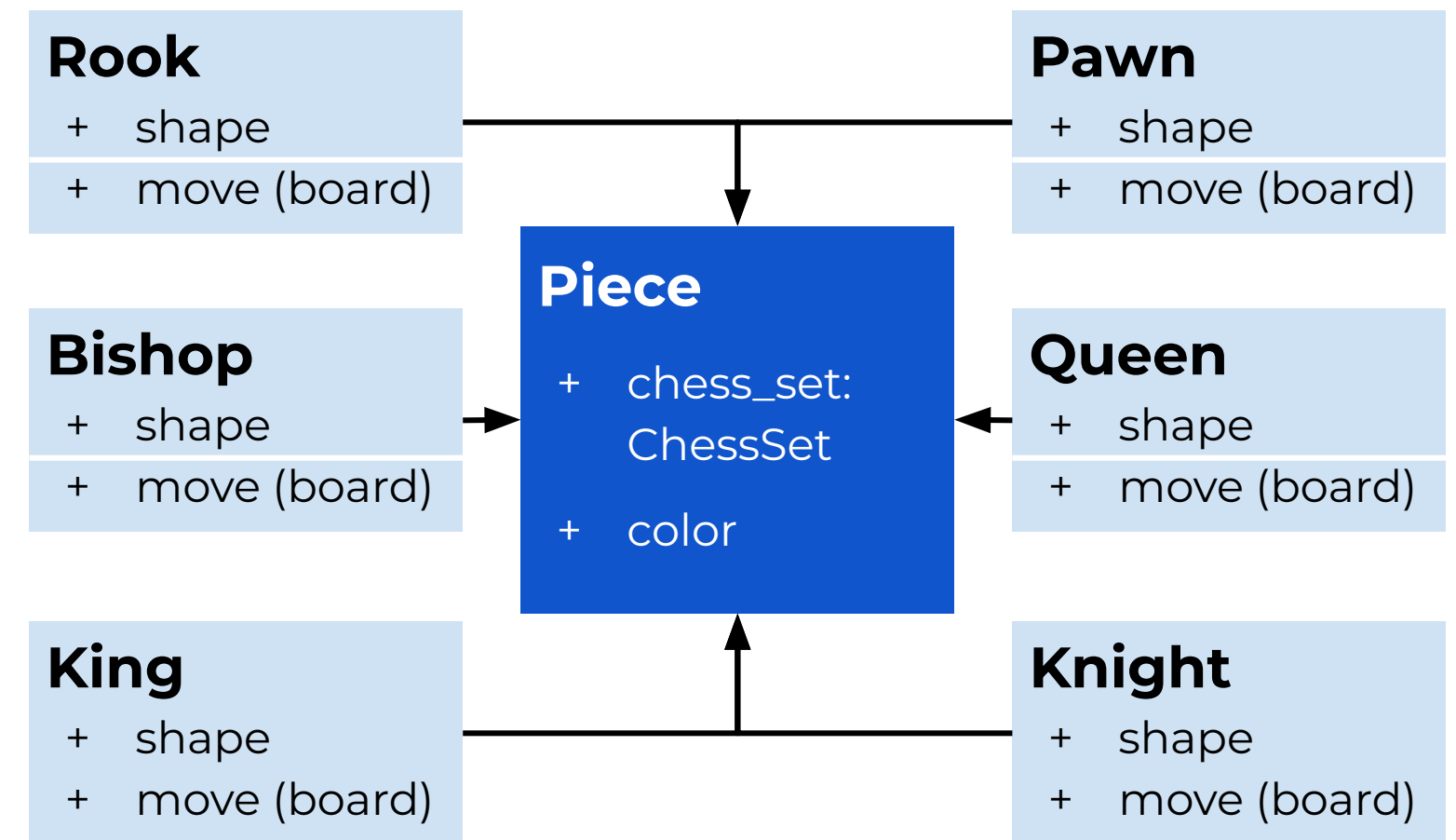


Что такое **наследование**:

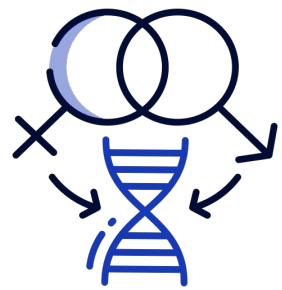
☆ Форма повторного использования программного обеспечения, при которой новый класс создается путем наследования членов существующего класса и расширения их новыми или измененными возможностями.

☆ Позволяет сэкономить время разработки, основывая новые классы на существующих, проверенных и отлаженных.

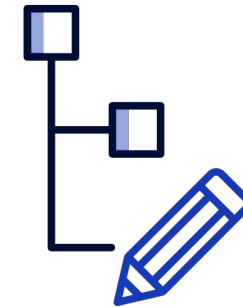
☆ Облегчает эффективное внедрение и сопровождение системы.



Что такое **наследование**



При создании нового класса можно указать, что он должен наследовать члены существующего класса



Новый класс является подклассом



Существующий класс является суперклассом

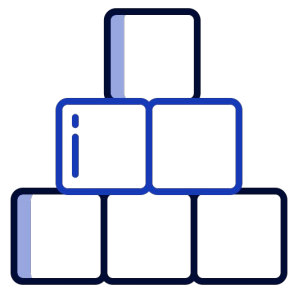


Каждый подкласс может быть суперклассом будущих подклассов

Что такое **наследование**



Подкласс может добавлять свои собственные поля и методы

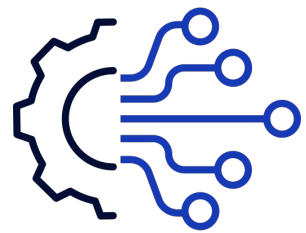


Подкласс более специфичен, чем его суперкласс, и представляет собой более специализированную группу объектов



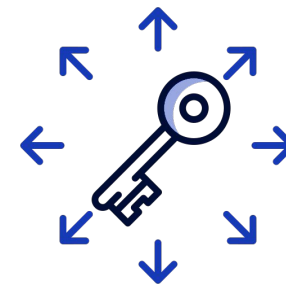
Подкласс демонстрирует поведение своего суперкласса плюс поведение, характерное для данного подкласса

Что такое **наследование**



Прямой суперкласс –

это суперкласс, от которого явно наследуется подкласс

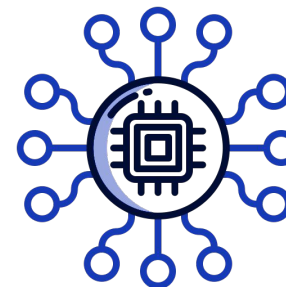


Каждый класс в Python прямо или косвенно расширяет (или «наследует от») класс object

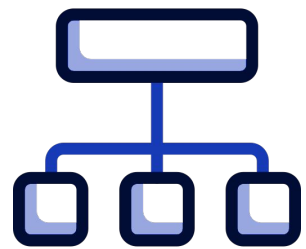


Косвенный суперкласс –

это любой класс, расположенный выше прямого суперкласса в иерархии классов



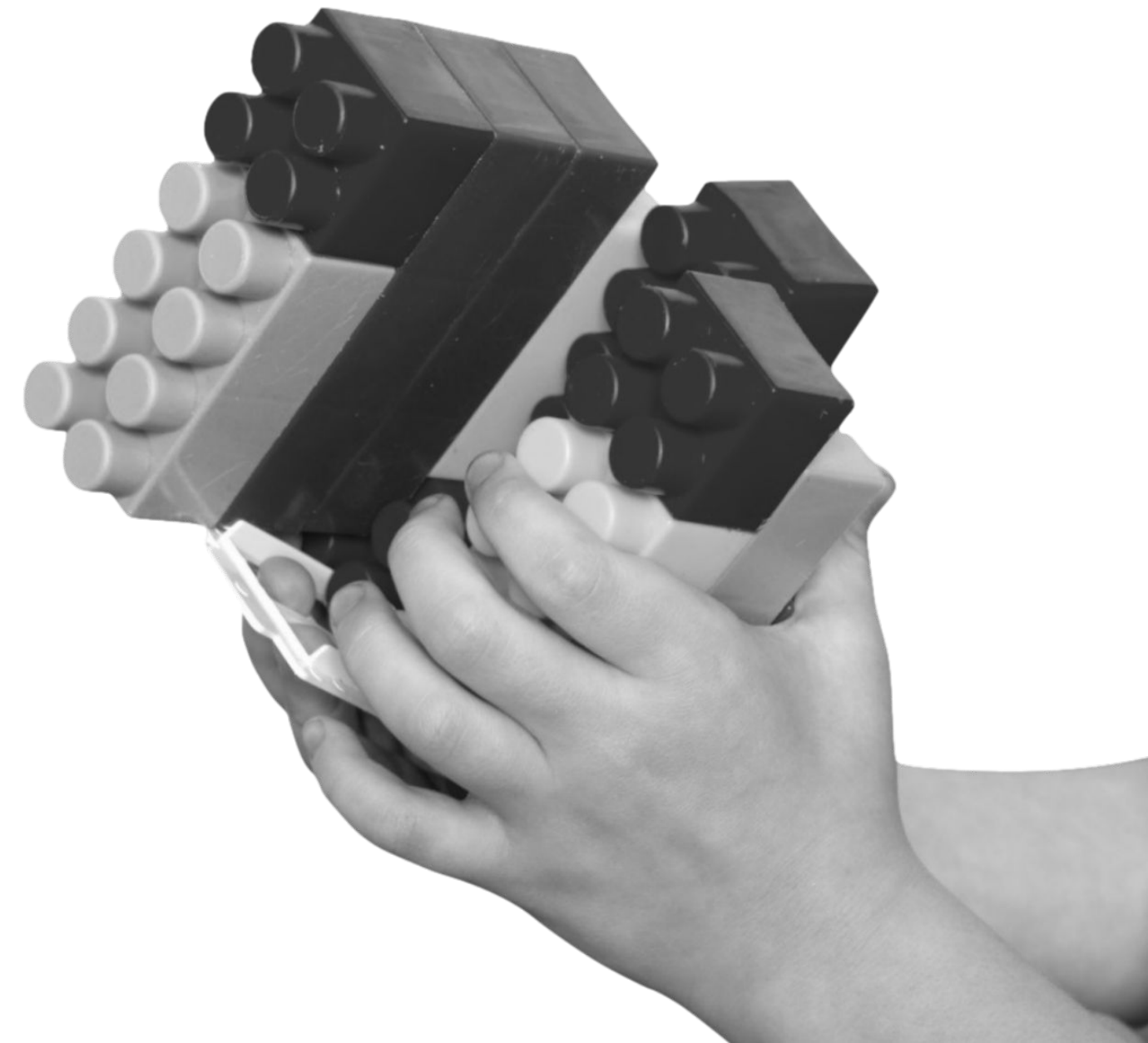
Python поддерживает множественное наследование, при котором каждый класс является производным от одного или более прямых суперклассов



Иерархия классов Python начинается с класса object (но указывать это необязательно)

Суперклассы и подклассы

- ☆ **Суперклассы** имеют тенденцию быть «более общими», а **подклассы** – «более конкретными».
- ☆ Поскольку каждый объект подкласса является объектом своего суперкласса, а один суперкласс может иметь много подклассов, набор объектов, представленных суперклассом, обычно больше, чем набор объектов, представленных любым из его подклассов.



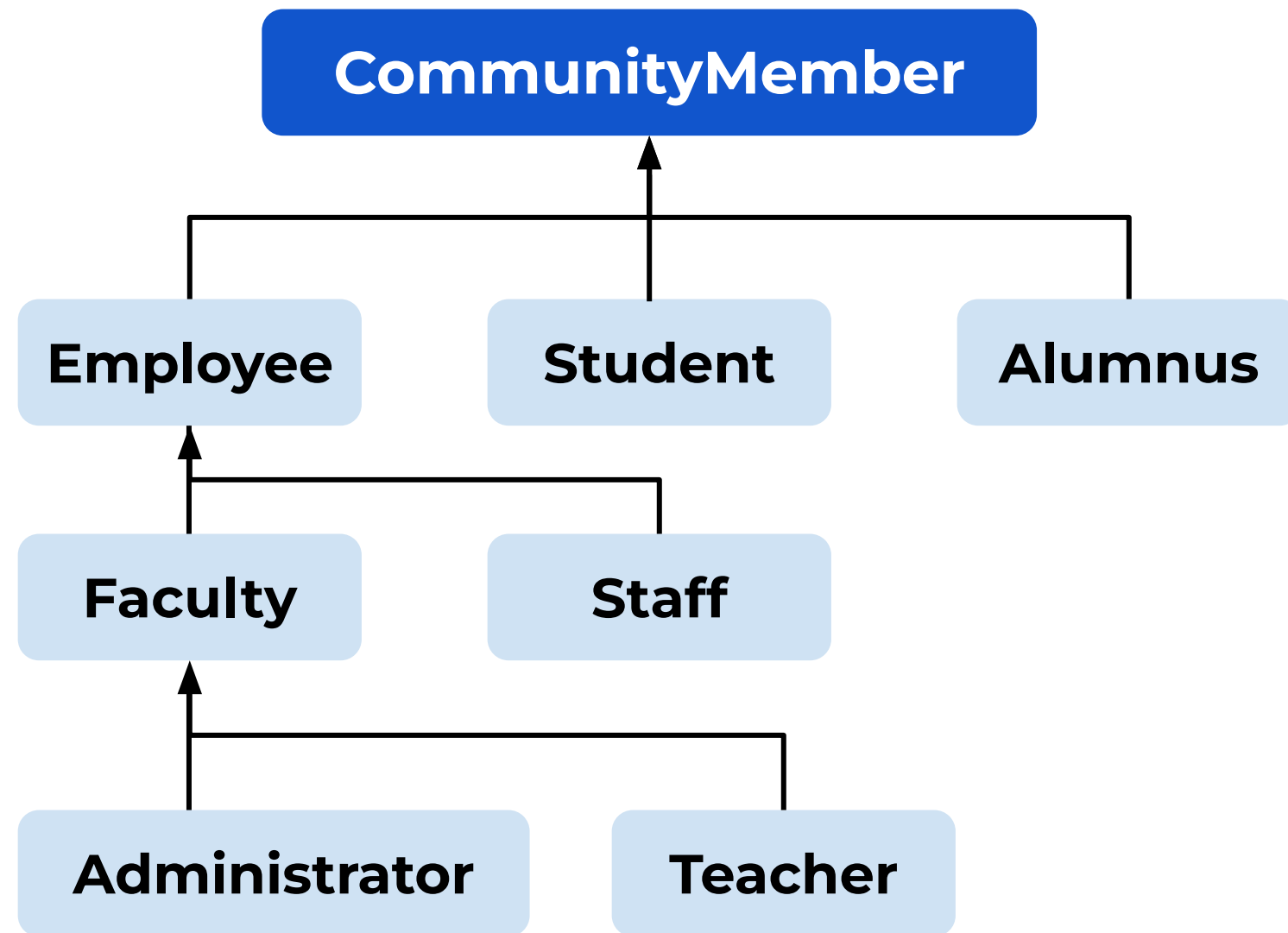
Суперклассы и подклассы

 Superclass	 Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

Fig. 9.1 | Inheritance examples

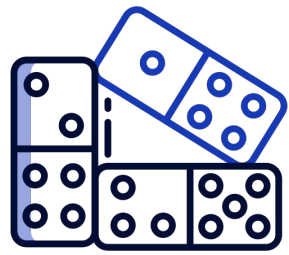
Суперклассы и подклассы

Fig. 9.2 | Inheritance hierarchy
for university **CommunityMember**



- ★ Следуйте по стрелкам вверх в иерархии классов
- ★ Сотрудник является членом сообщества, преподаватель – член факультета
- ★ **CommunityMember** является прямым суперклассом классов **Employee**, **Student** и **Alumnus** и косвенным суперклассом всех остальных классов на диаграмме

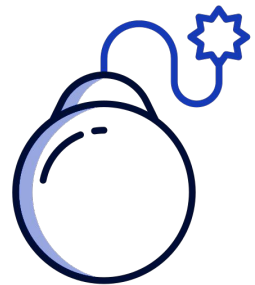
Суперклассы и подклассы



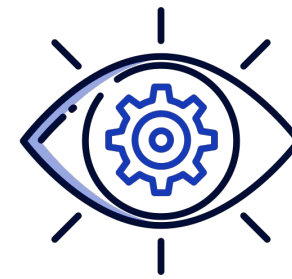
Объекты всех классов, которые расширяют общий суперкласс, могут рассматриваться как объекты этого суперкласса



Даже если метод суперкласса подходит для подкласса, подклассу часто требуется адаптированная версия метода



Проблемы наследования



Подкласс может переопределить (пересмотреть) метод суперкласса с соответствующей реализацией



Подкласс может унаследовать методы, которые ему не нужны или не должны быть у него

Как наследование реализовано в Python

```
class Animal(object):
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def eat(self):
        print(f"{self.name} is eating.")

    def sleep(self):
        print(f"{self.name} is sleeping.")

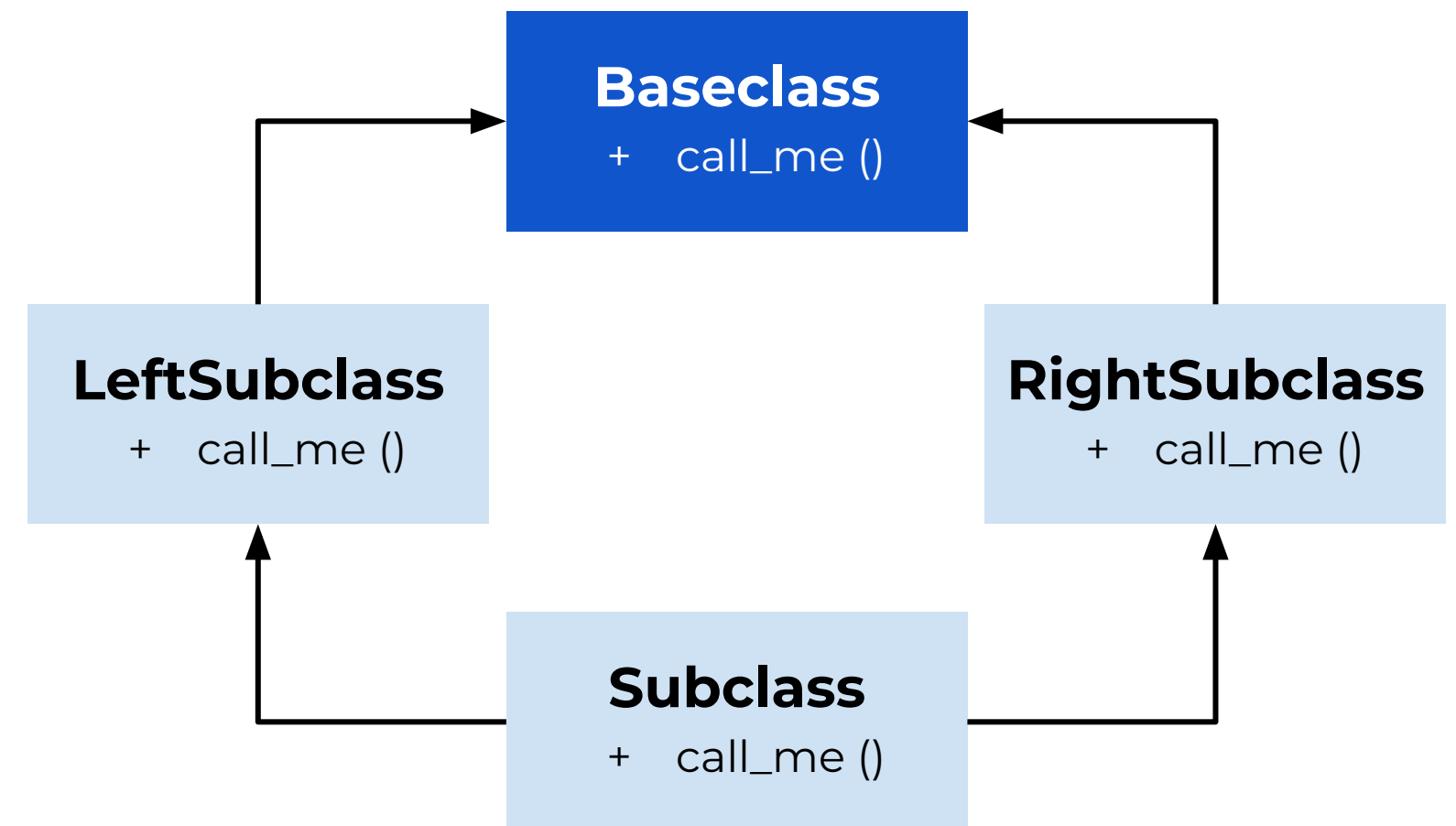
class Cat(Animal):
    def __init__(self, name, species, color):
        super().__init__(name, species)
        self.color = color

    def meow(self):
        print(f"{self.name} is meowing.")
```



Порядок наследования

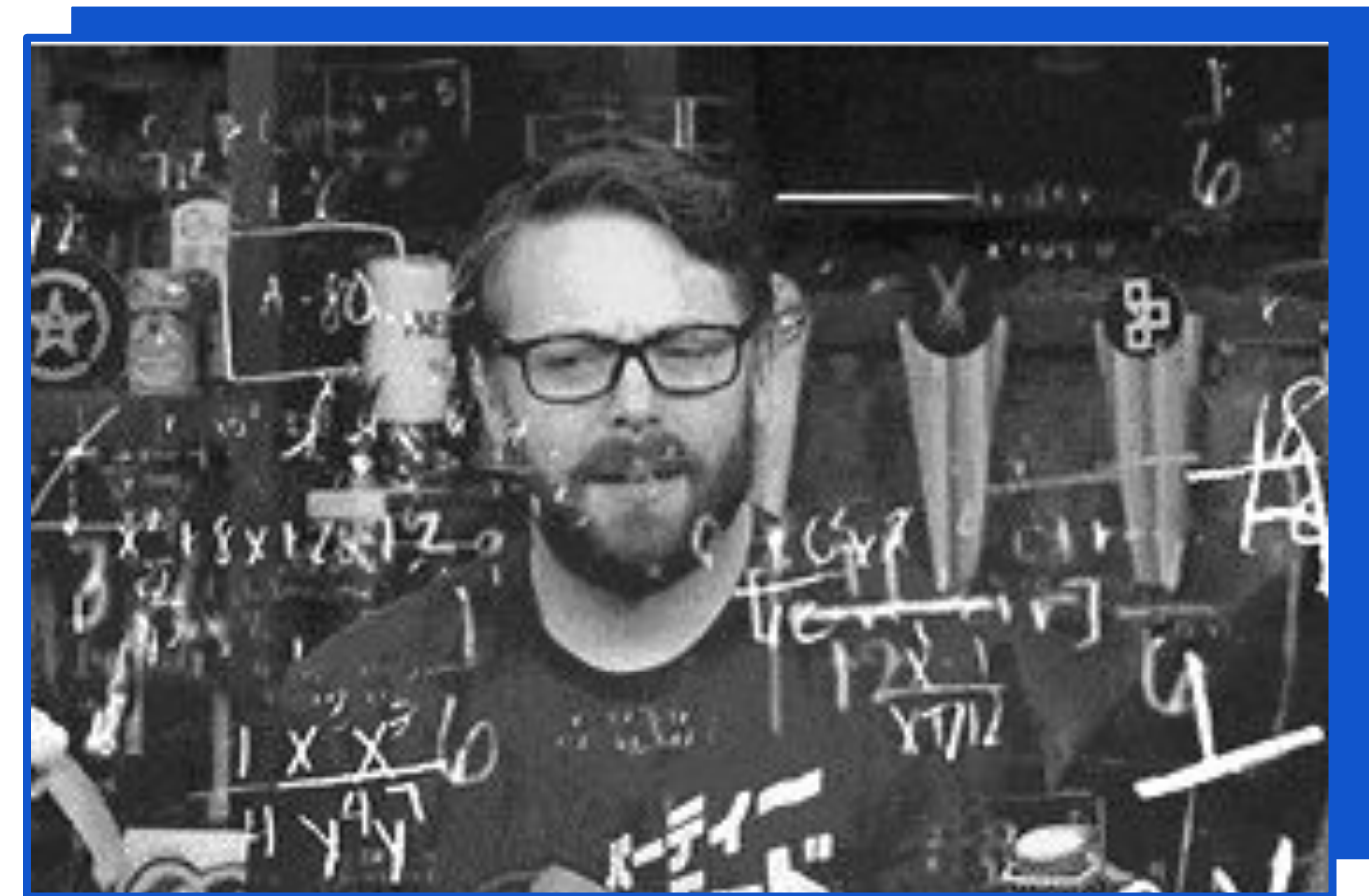
- ☆ Python поддерживает наследование, позволяя одному классу наследовать свойства и методы от другого класса
- ☆ Наследование достигается путем наследования подкласса от суперкласса
- ☆ Однако порядок, в котором наследуются классы, может повлиять на поведение подкласса



Порядок наследования

- 1 В Python порядок наследования задается с помощью кортежа, называемого порядком разрешения методов (MRO).
- 2 MRO определяет порядок, в котором Python ищет методы и свойства при разрешении ссылок на атрибуты.
- 3 Для определения MRO Python использует алгоритм поиска в глубину.

**Trying to understand
the Python MRO**



Порядок наследования

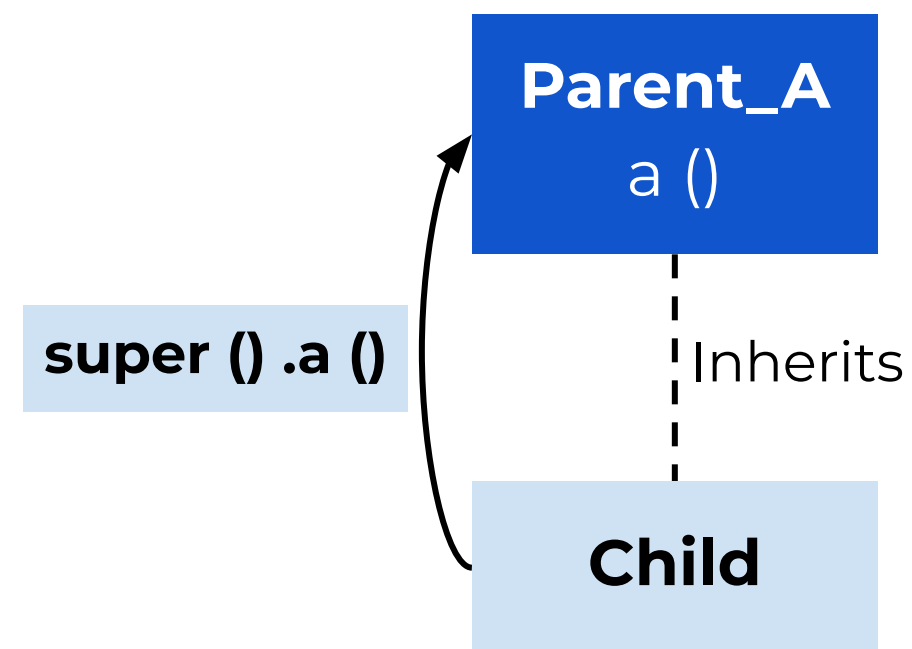
4 Чтобы указать порядок наследования в Python, можно использовать функцию **"super"**.

5 Функция **"super"** принимает два аргумента: подкласс и экземпляр подкласса.

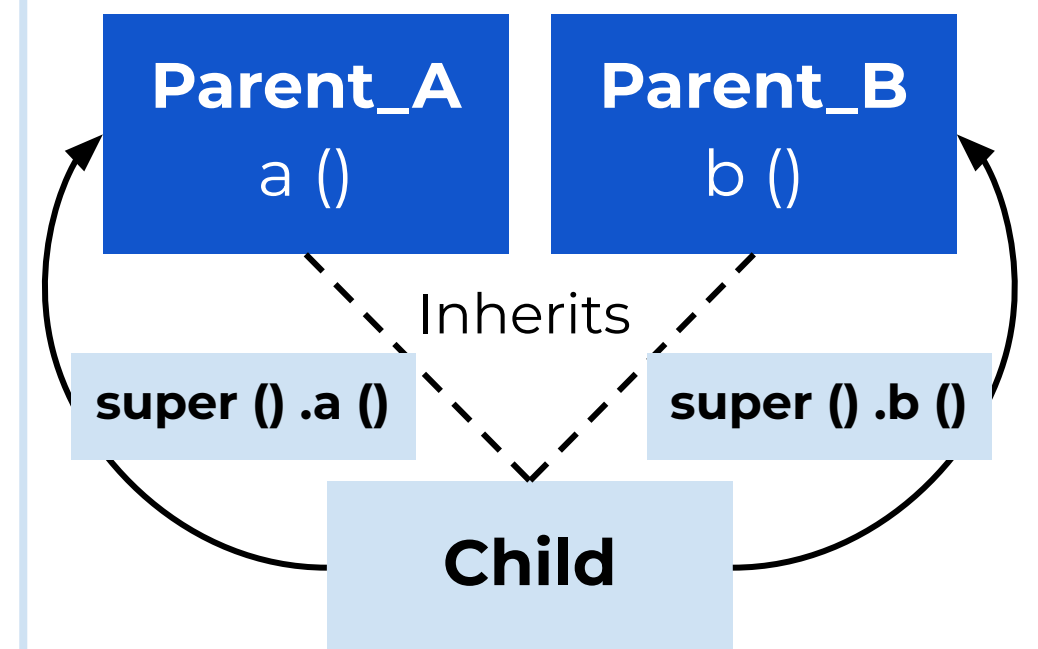
6 Вызывая **"super()"** с этими аргументами, вы можете указать порядок, в котором Python должен искать методы и свойства.



Single Inheritance



Multiple Inheritance



Порядок наследования

- 7 Если вы не укажете порядок наследования с помощью функции "**super**", Python будет использовать MRO по умолчанию, который определяется порядком, в котором базовые классы перечислены в определении подкласса.
- 8 Это может привести к неожиданному поведению, особенно при работе с множественным наследованием.



Порядок наследования

- 9 Чтобы избежать проблем с множественным наследованием, лучше всего использовать функцию **"super"** для явного указания порядка наследования в Python.
- 10 Этим вы обеспечите предсказуемость поведения вашего кода и правильное разрешение любых потенциальных конфликтов.



Порядок наследования

В этом примере у нас есть три класса: **A**, **B** и **C**.

C наследует от **A** и **B**.

Мы хотим установить приоритет метода `__init__()` **A** над **B**, поэтому мы используем функцию `super()`, чтобы указать, что мы хотим сначала вызвать метод `__init__()` **A**.

В методе `__init__()` в **C** мы вызываем `super().__init__()`. Это вызывает метод `__init__()` следующего класса в порядке разрешения методов (MRO) после **C**, которым является **A**. Это инициализирует атрибут `message` класса **A**, который затем выводится на консоль.

```
class A:
    def __init__(self):
        self.message = "Hello from A!"

class B:
    def __init__(self):
        self.message = "Hello from B!"

class C(A, B):
    def __init__(self):
        super().__init__() # Call the __init__() method of A
        print(self.message) # Prints "Hello from A!"

c = C()
```

Результатом выполнения этого кода будет:

```
Hello from A!
```

Порядок наследования

- 11 В целом, порядок наследования в Python определяется порядком разрешения методов (MRO), который можно задать с помощью функции **"super"**.
- 12 Важно тщательно учитывать порядок наследования при работе с множественным наследованием, чтобы избежать неожиданного поведения и конфликтов.
- 13 Используя функцию **"super"** для явного указания порядка наследования, вы можете обеспечить хорошую организацию, сопровождаемость и предсказуемость вашего кода.



Изменение порядка наследования

- 1 Другой способ изменить порядок наследования в Python – использовать атрибут **"bases"**.
- 2 Атрибут **"bases"** представляет собой кортеж, содержащий базовые классы подкласса в том порядке, в котором они были перечислены в определении подкласса.
- 3 Вы можете изменить порядок кортежа **"bases"**, чтобы изменить порядок наследования.
- 4 Чтобы изменить порядок наследования с помощью атрибута **"bases"**, вы можете создать новый кортеж с желаемым порядком базовых классов и присвоить его атрибуту **"bases"** подкласса.



Например, вы можете создать новый MRO с порядком [C, B, A] следующим образом: **C.bases = (B, A)**

Изменение порядка наследования

```
class A:
    def __init__(self):
        self.message = "Hello from A!"

class B:
    def __init__(self):
        self.message = "Hello from B!"

class C(A, B):
    def __init__(self):
        super().__init__() # Call the __init__() method of A
        print(self.message) # Prints "Hello from A!"

# Change the order of inheritance for C
C.__bases__ = (B, A)

c = C()
```



Изменение порядка наследования

Результатом выполнения этого кода будет:

Hello from B!



Обратите внимание, что изменение порядка наследования с помощью **__bases__** может быть полезно в некоторых ситуациях, но в целом считается менее безопасным и менее удобным, чем использование **super()**. Кроме того, изменение порядка наследования может иметь неожиданные последствия, поэтому важно тщательно тестировать код при внесении подобных изменений.

Изменение порядка наследования

- 5 В целом, изменить порядок наследования в Python можно с помощью функции **"super"** или атрибута **"bases"**.
- 6 Изменяя порядок наследования, вы можете отдавать предпочтение одним методам и свойствам перед другими и избегать конфликтов между базовыми классами.
- 7 Важно тщательно продумать последствия изменения порядка наследования и тщательно протестировать свой код, чтобы убедиться, что он ведет себя так, как ожидается.

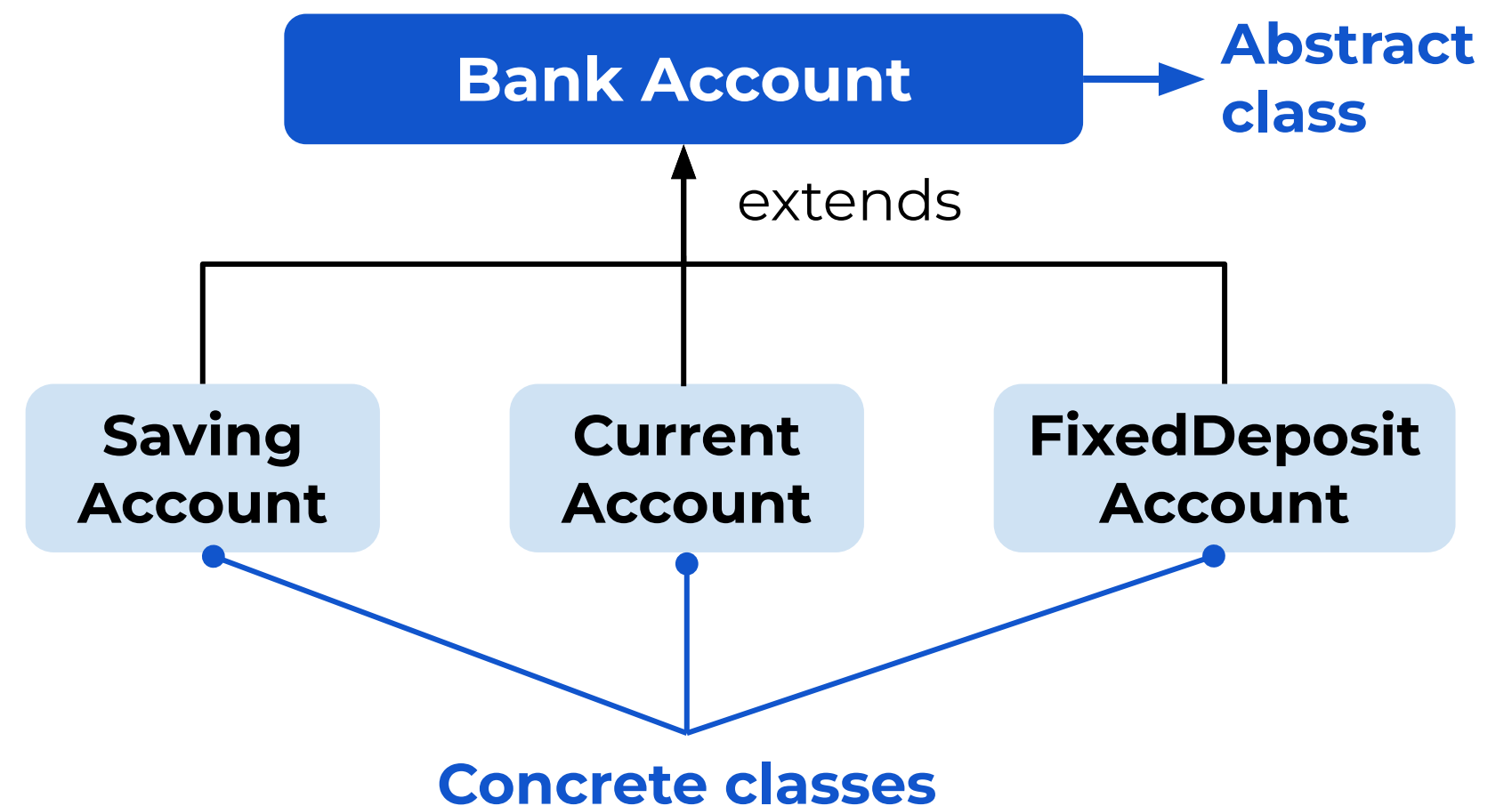


Абстрактные методы и классы

Абстрактный класс – это класс, который не может быть создан напрямую.

Абстрактные классы можно использовать только в качестве базового класса для других классов, которые их расширяют.

Абстрактные классы часто используются для определения общего интерфейса для группы родственных классов.



Абстрактные методы и классы

Абстрактный метод – это метод, который объявлен в абстрактном классе, но не имеет реализации.

Абстрактные методы должны быть реализованы любым конкретным (неабстрактным) классом, который расширяет абстрактный класс.

Абстрактные методы часто используются для определения набора методов, которые должна реализовывать группа родственных классов.



Абстрактные методы и классы

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def speak(self):
        pass
```

```
class Cat(Animal):
    def speak(self):
        return "Meow"
```

В этом примере мы определяем абстрактный класс **Animal**, который имеет один абстрактный метод **speak()**. Декоратор **@abstractmethod** указывает, что метод **speak()** не имеет реализации и должен быть реализован любым конкретным классом, который расширяет **Animal**.

В этом примере мы определяем конкретный класс **Cat**, который расширяет абстрактный класс **Animal** и реализует метод **speak()**. Поскольку **Cat** реализует метод **speak()**, он может быть создан и использован как любой другой класс.

Абстрактные методы и классы

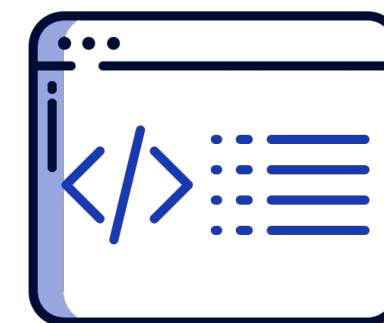
Преимущества абстрактных классов и методов



Предоставляет
возможность определить
общий интерфейс
для группы родственных
классов



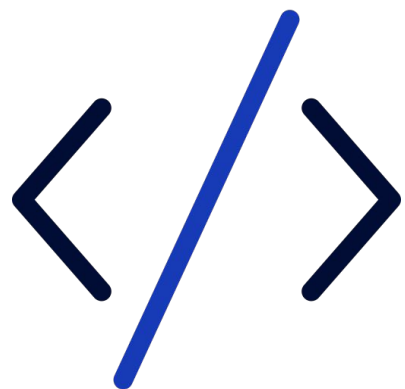
Гарантирует,
что все классы, расширяющие
абстрактный класс, реализуют
необходимые методы



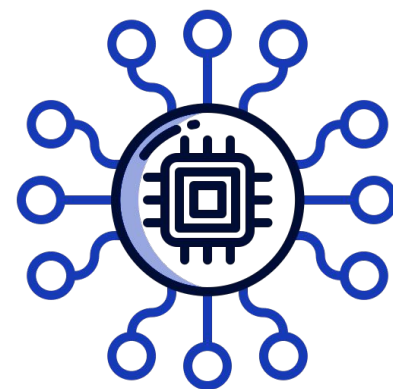
Помогает сократить
дублирование кода за счет
определения общей
функциональности
в абстрактном классе

Абстрактные методы и классы

Заключение

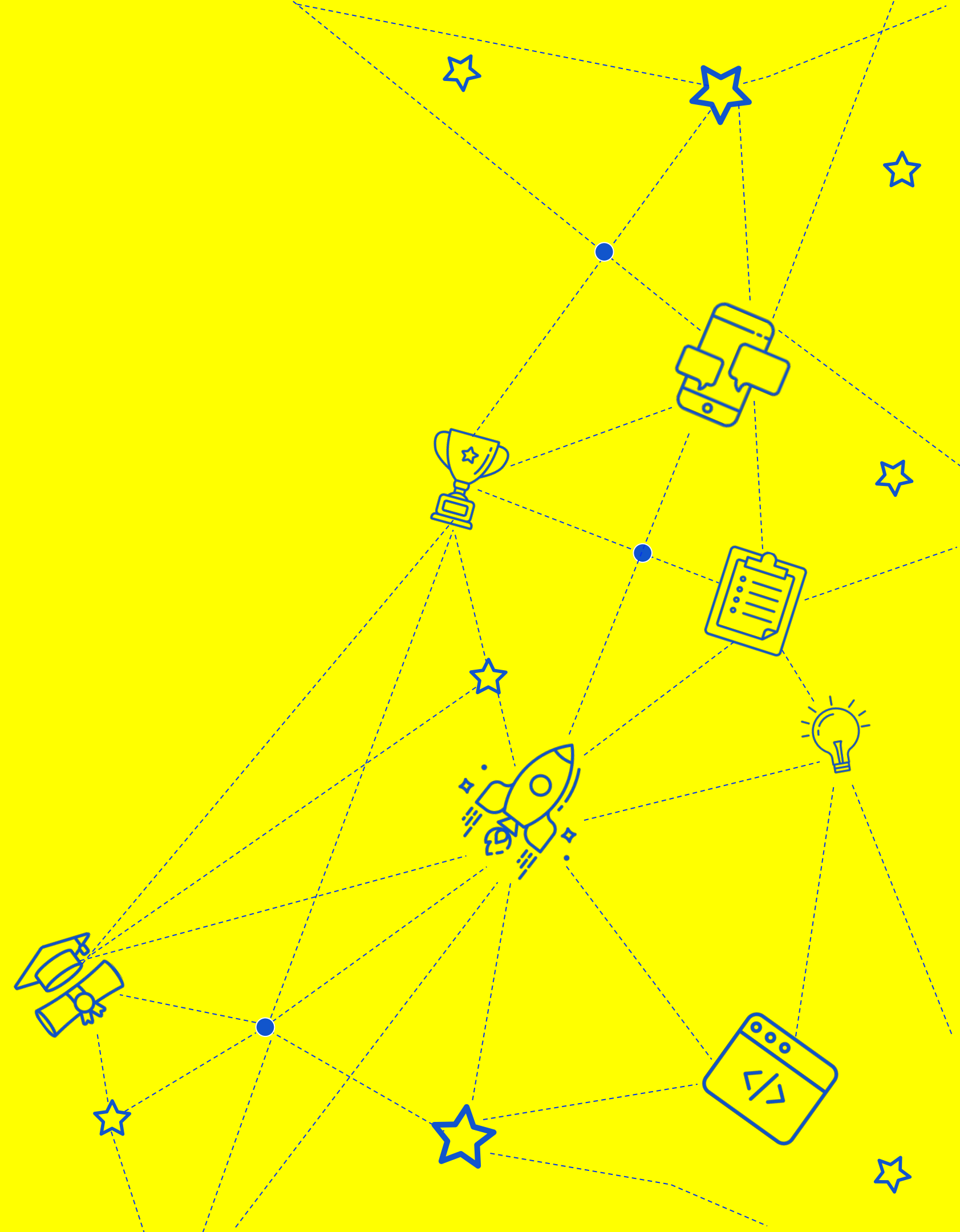


Абстрактные классы и методы – это мощная функция Python, которая может помочь вам писать более удобный и расширяемый код



Используйте **абстрактные классы и методы**, чтобы определить общий интерфейс для группы связанных классов и убедиться, что все классы, расширяющие ваш абстрактный класс, реализуют необходимые методы

Инкапсуляция



Инкапсуляция

Что изучим

- 1 Что такое инкапсуляция
- 2 Модификаторы доступа в инкапсуляции
- 3 Инкапсуляция в Python

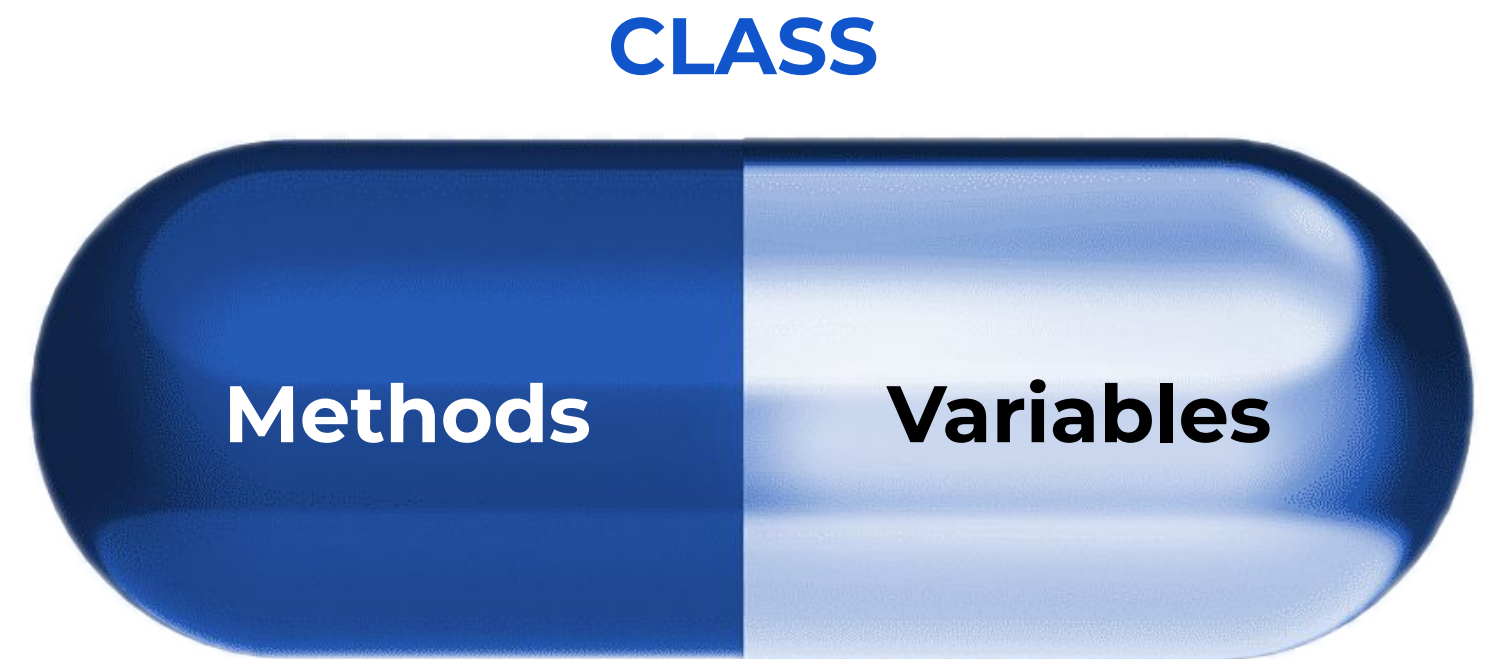


Что такое **инкапсуляция**

Инкапсуляция – это фундаментальная концепция объектно-ориентированного программирования (ООП).

Она относится к практике сокрытия внутренних деталей реализации объекта и раскрытия только публичного интерфейса.

Это помогает защитить объект от внешних манипуляций и обеспечивает согласованность его поведения.



Модификаторы доступа в инкапсуляции

Модификаторы доступа используются для управления видимостью переменных и методов внутри класса.

Они указывают, какие части класса доступны извне класса.

Общие модификаторы доступа включают:

- ★ **private**
- ★ **protected**
- ★ **public**

Access Specifiers	Same Class	Same Package	Derived Class	Other Class
Public	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	No
Private	Yes	No	No	No

Модификаторы доступа в инкапсуляции



Модификатор **приватного доступа**

- ★ Доступ к приватным переменным и методам возможен только в пределах класса, в котором они определены.
- ★ Во многих языках программирования они обозначаются знаком подчеркивания (_).
- ★ Это помогает гарантировать, что внутренние детали реализации скрыты от внешнего кода

Модификаторы доступа в инкапсуляции



Модификатор **защищенного доступа**

- ★ Доступ к защищенным переменным и методам возможен в пределах класса, в котором они определены, и любых подклассов.
- ★ В некоторых языках программирования они обозначаются знаком подчеркивания (**_**), в других – ключевым словом **"protected"**.
- ★ Это позволяет подклассам получать доступ и изменять детали реализации, но при этом защищает их от внешнего кода.

Модификаторы доступа в инкапсуляции

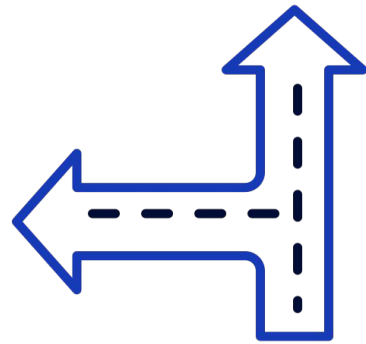


Модификатор **публичного доступа**

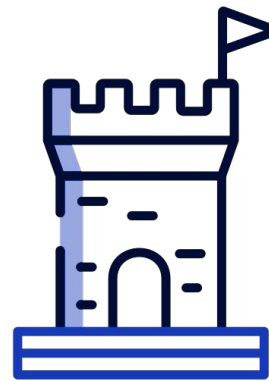
- ★ К общедоступным переменным и методам можно обращаться из любой точки программы.
- ★ Они не имеют специального синтаксиса или ключевых слов, связанных с ними.
- ★ Это позволяет внешнему коду взаимодействовать с объектом контролируемым и последовательным образом.

Модификаторы доступа в инкапсуляции

Преимущества инкапсуляции



Инкапсуляция помогает
обеспечить предсказуемое
и последовательное
поведение объектов



Она делает код
более модульным,
надежным и удобным
для сопровождения



Скрывая детали реализации
и раскрывая чистый публичный
интерфейс, она снижает
вероятность ошибок и облегчает
изменение реализации
без влияния на другие части
программы

Инкапсуляция в Python

Инкапсуляция – это фундаментальная концепция объектно-ориентированного программирования (ООП).

Она относится к практике сокрытия внутренних деталей реализации объекта и раскрытия только публичного интерфейса.

Это помогает защитить объект от внешних манипуляций и обеспечивает согласованность его поведения.

```
class Demo:
    x = 20
    _y = 30
    __z = 40

    def method1(self):
        print('Hello I am public method')

    def __method2(self):
        print('Hello I am private method')

    def method3(self):
        self.__method2()

d = Demo()
d.method1()
d.__method2()
```

Инкапсуляция в Python

Пример инкапсуляции в Python

```
class BankAccount:
    def __init__(self, initial_balance):
        self._balance = initial_balance

    def deposit(self, amount):
        self._balance += amount

    def withdraw(self, amount):
        if amount > self._balance:
            print("Insufficient funds")
        else:
            self._balance -= amount

    def get_balance(self):
        return self._balance
```

Рассмотрим пример класса, представляющего банковский счет.

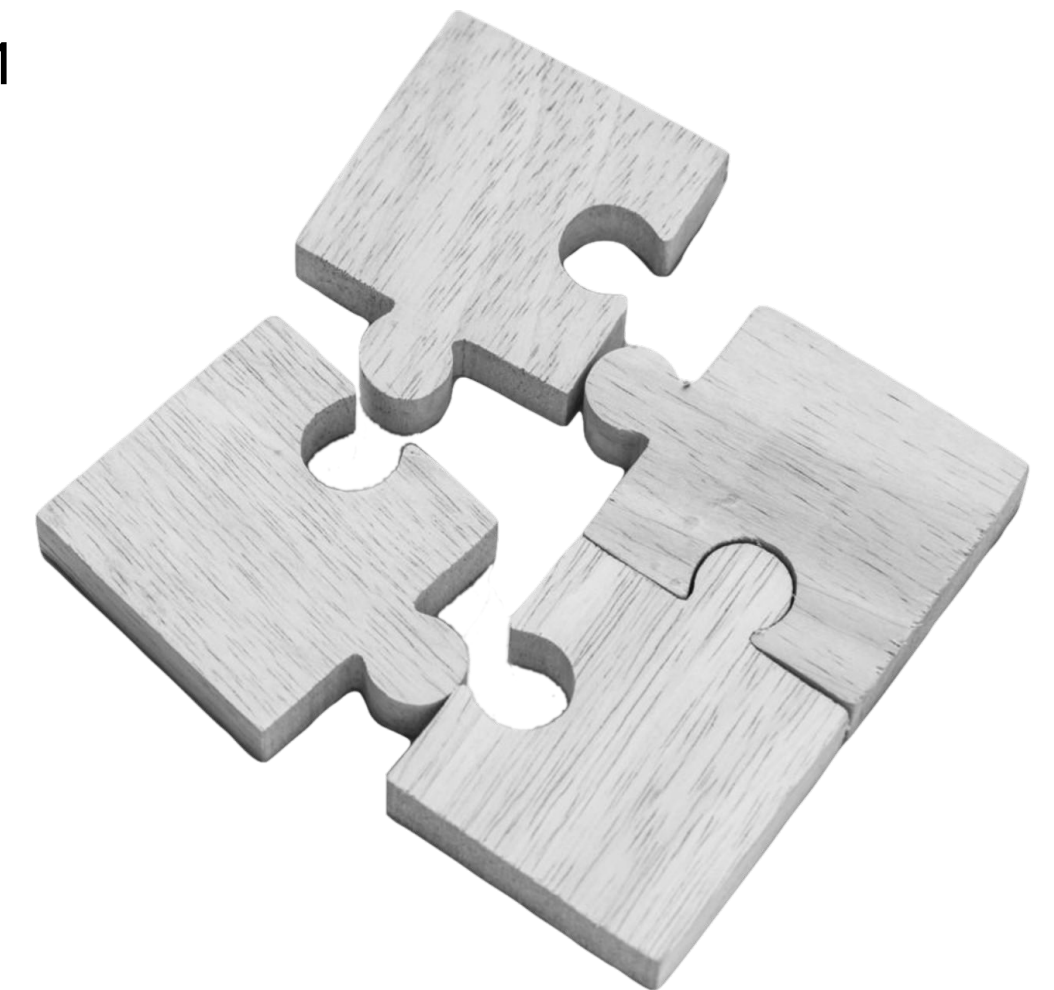
Мы хотим инкапсулировать баланс счета так, чтобы он не мог быть непосредственно изменен извне класса.

Мы можем достичь этого, используя приватную переменную экземпляра и предоставляя публичные методы для взаимодействия с ней.

Инкапсуляция в Python

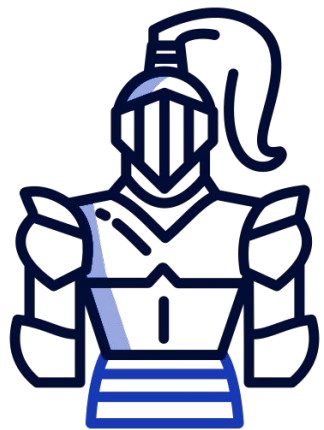
Использование класса **BankAccount**

- 1** Мы можем создать экземпляр класса **BankAccount** и взаимодействовать с ним, используя его публичные методы.
- 2** Мы не можем получить прямой доступ к переменной баланса извне класса.
- 3** Это помогает гарантировать, что доступ к балансу и его изменение всегда осуществляются последовательным образом.

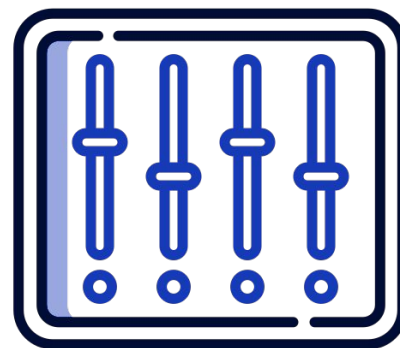


Инкапсуляция в Python

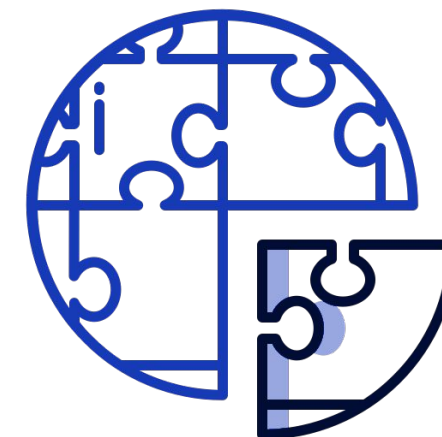
Заключение



Инкапсуляция – это мощная техника, которая помогает сделать наш код более модульным, надежным и удобным для сопровождения.

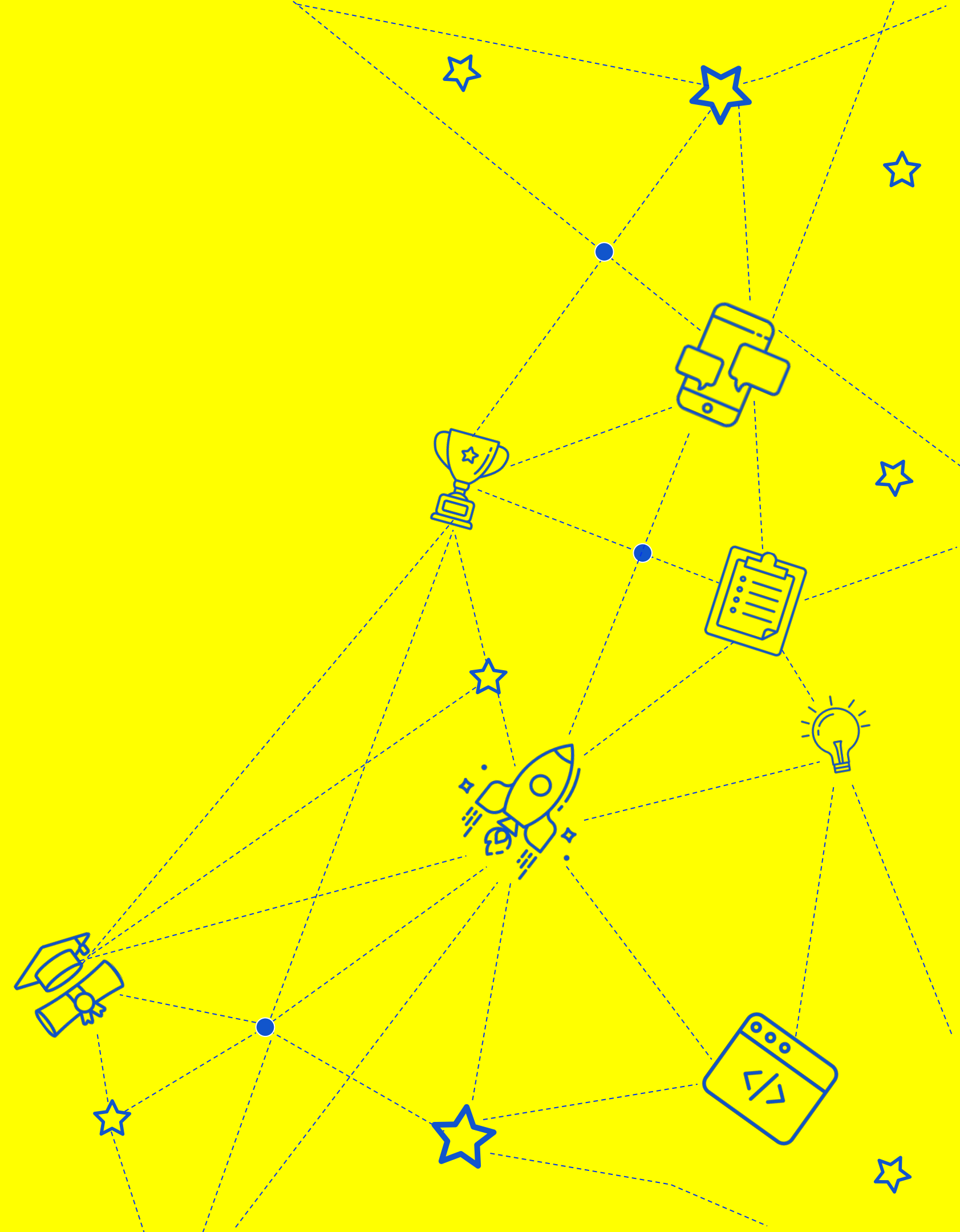


Скрывая детали реализации и раскрывая чистый публичный интерфейс, мы можем гарантировать, что объекты ведут себя предсказуемо и последовательно.



Отсутствие в Python встроенных модификаторов доступа означает, что для достижения инкапсуляции мы должны полагаться на соглашения об именовании, но это малая цена за преимущества, которые она дает.

Полиморфизм



Полиморфизм

Что изучим

- 1 Что такое полиморфизм
- 2 Типы полиморфизма
- 3 Перезагрузка методов
- 4 Переопределение методов
- 5 Использование полиморфизма

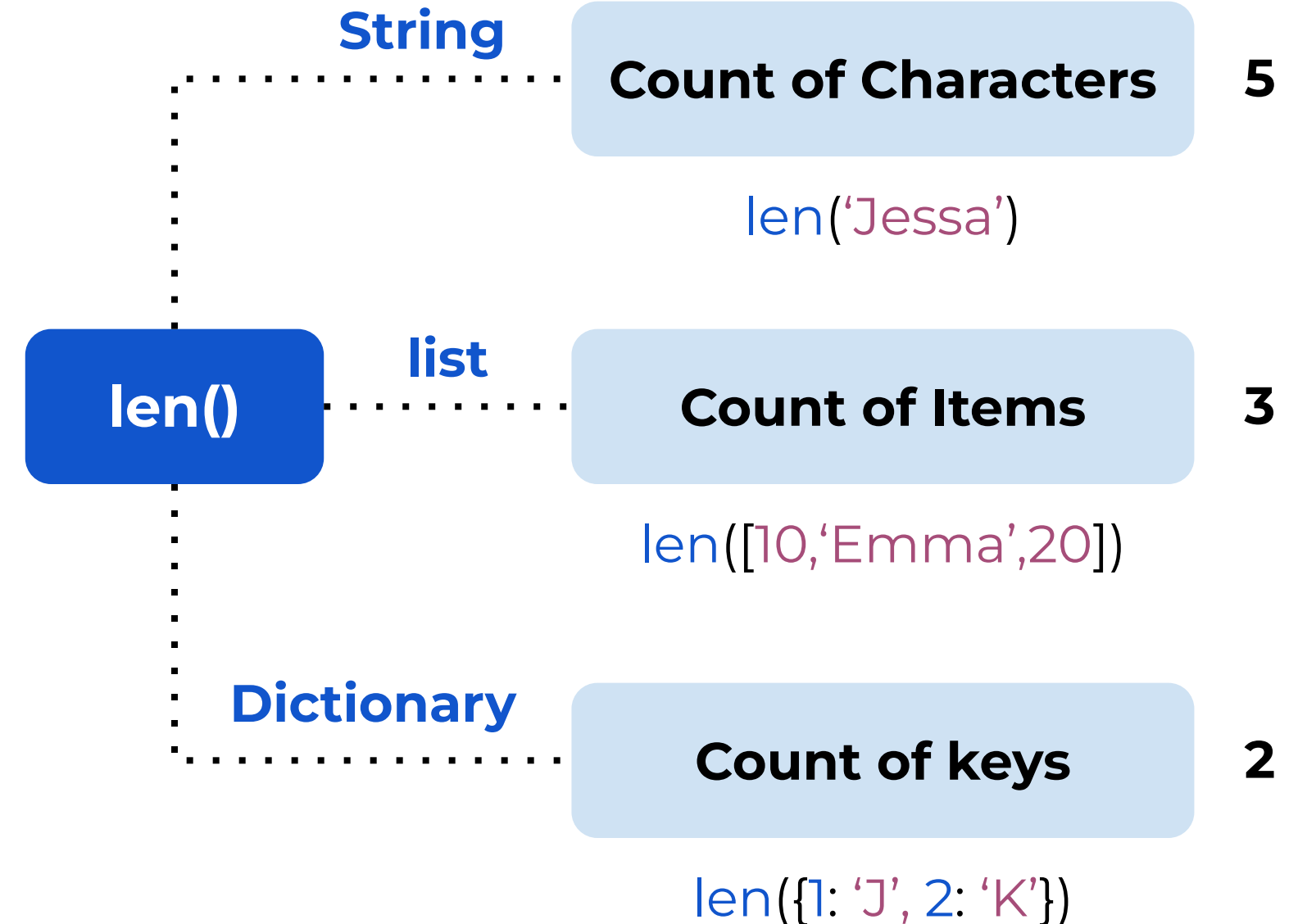


Что такое полиморфизм

Полиморфизм – это фундаментальная концепция объектно-ориентированного программирования (ООП).

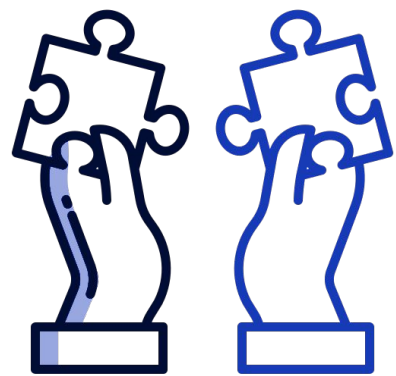
Он относится к способности объектов различных классов рассматриваться так, как если бы они были объектами одного и того же класса.

Это позволяет нам писать более гибкий и многократно используемый код.



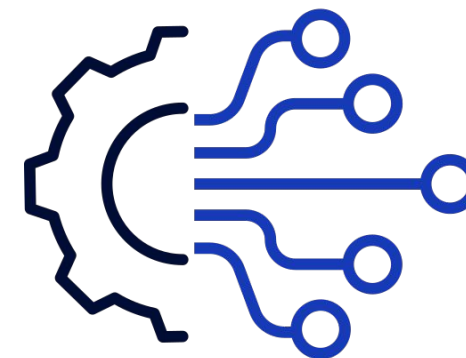
Типы полиморфизма

В ООП существует два типа полиморфизма:



Полиморфизм во время компиляции

относится к возможности использовать различные методы с одинаковым именем и различными списками параметров



Полиморфизм во время выполнения

относится к возможности использовать один и тот же метод с различными объектами, которые могут иметь различные реализации этого метода

Перезагрузка методов

Перегрузка методов – это форма полиморфизма во время компиляции.

Она позволяет нам определить несколько методов с одинаковым именем, но разными списками параметров.

Подходящий метод для вызова определяется во время компиляции на основе количества и типов предоставленных аргументов.



Переопределение методов

Переопределение методов – это форма полиморфизма во время выполнения.

Оно позволяет подклассу обеспечить собственную реализацию метода, который уже определен в суперклассе.

Подходящий метод для вызова определяется во время выполнения на основе типа объекта.

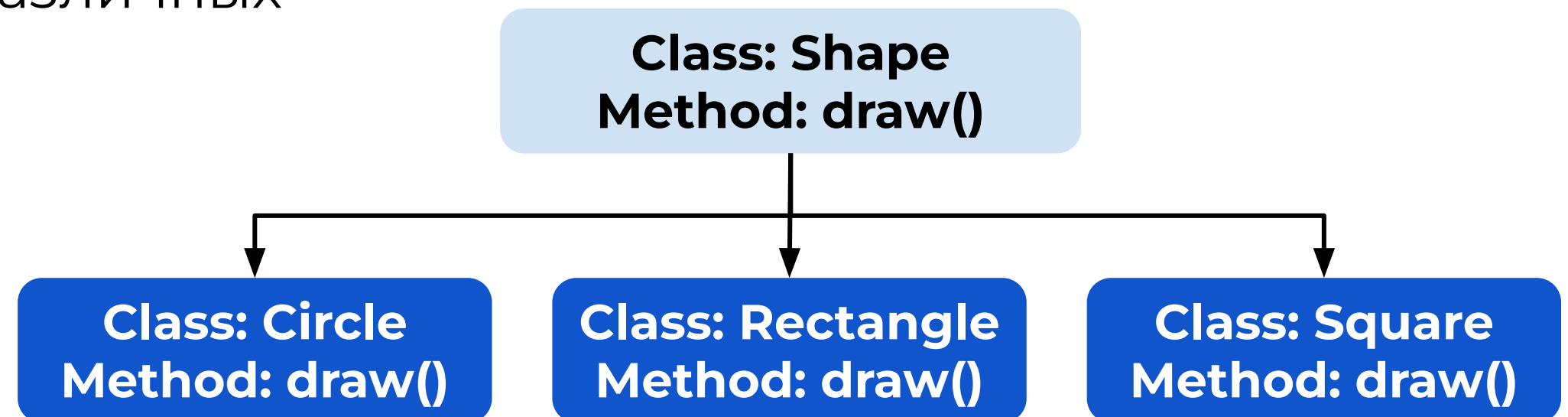


Использование полиморфизма

Рассмотрим пример иерархии классов, представляющих различные геометрические фигуры.

Мы можем определить метод **"area"** в суперклассе, который переопределяется в подклассах для производства различных звуков.

Пример полиморфизма



Использование полиморфизма

```
class Shape:
    def area(self):
        pass # abstract method that will be overridden in subclasses

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

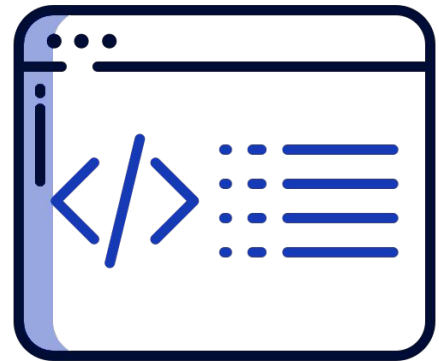
# create instances of the Square and Circle classes
s = Square(5)
c = Circle(2)

# call the area method on both objects, even though they are of different classes
print("Area of square:", s.area())
print("Area of circle:", c.area())
```

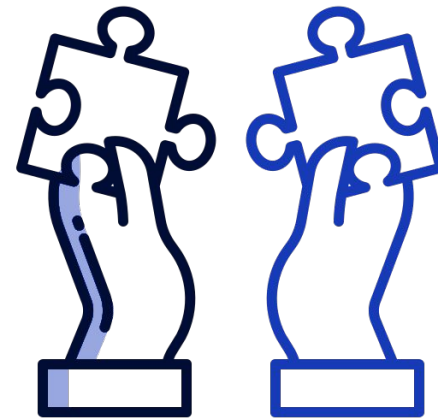
В этом примере у нас есть суперкласс **Shape** с абстрактным методом **area**. Этот метод будет переопределен в подклассах, которыми являются **Square** и **Circle**. Каждый подкласс имеет свою собственную реализацию метода **area**, основанную на его конкретной форме.

Затем мы создадим экземпляры классов **Square** и **Circle** и вызовем метод **area** для каждого объекта. Несмотря на то, что объекты принадлежат к разным классам, мы можем обращаться с ними так, как будто это объекты одного класса (т.е. **Shape**), и вызывать один и тот же метод для обоих. **Это пример полиморфизма во время выполнения.**

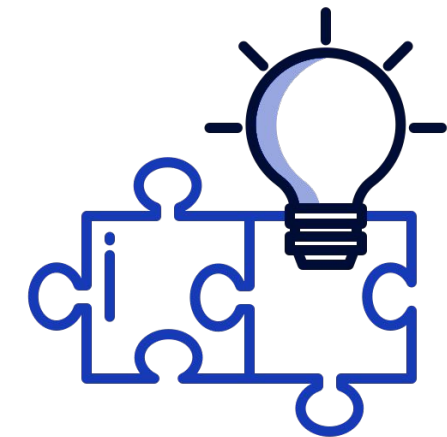
Использование полиморфизма



Мы можем создать
экземпляры классов
Square и Circle
и вызвать их методы **area**



Несмотря на то, что объекты
принадлежат к разным
классам, мы можем обращаться
с ними так, как если бы они
были объектами одного класса
(Shape)



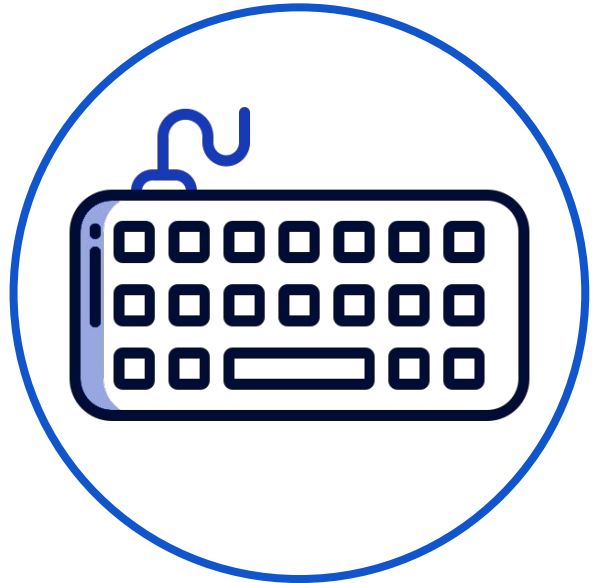
Это позволяет нам
писать более гибкий
и многократно
используемый код

Использование полиморфизма: **выводы**

- ☆ **Полиморфизм** – это мощная концепция ООП, которая позволяет нам писать более гибкий и многократно используемый код.
- ☆ **Существует два типа полиморфизма:** во время компиляции и во время выполнения.
- ☆ **Перегрузка методов и переопределение методов** – две распространенные формы полиморфизма.
- ☆ Рассматривая объекты разных классов как объекты одного класса, мы можем писать более модульный и простой в сопровождении код.

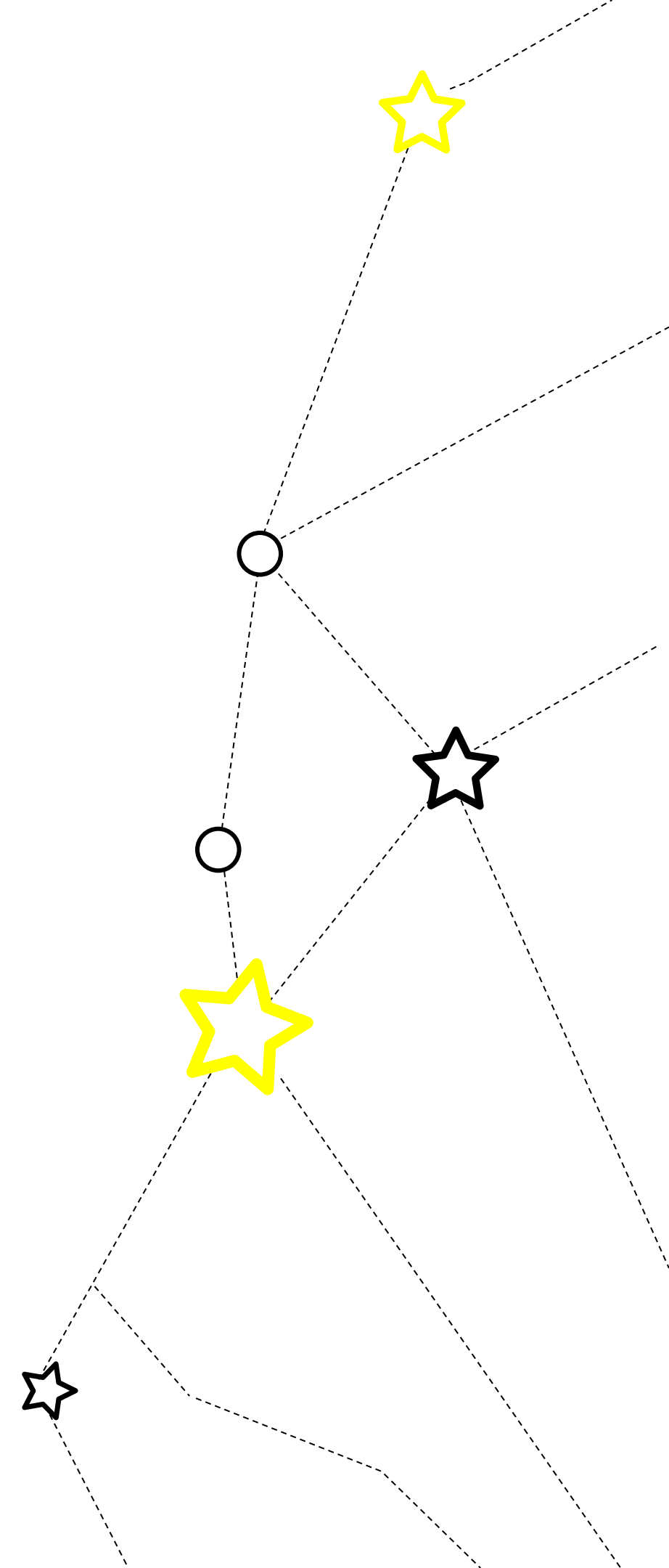


Домашнее задание



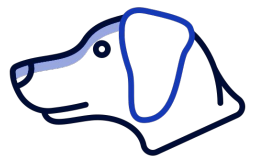
Создайте иерархию классов для зоомагазина. Иерархия должна включать базовый класс **Pet**, который имеет следующие атрибуты и методы:

- ★ **имя (строка):** имя питомца
- ★ **вид (строка):** вид животного
- ★ **возраст (int):** возраст питомца
- ★ **пол (строка):** пол питомца
- ★ **get_info() (метод):** выводит имя, вид, возраст и пол питомца.



Домашнее задание

Далее создайте два подкласса **Pet** под названием **Dog** и **Cat**, каждый со своими специфическими атрибутами и методами:



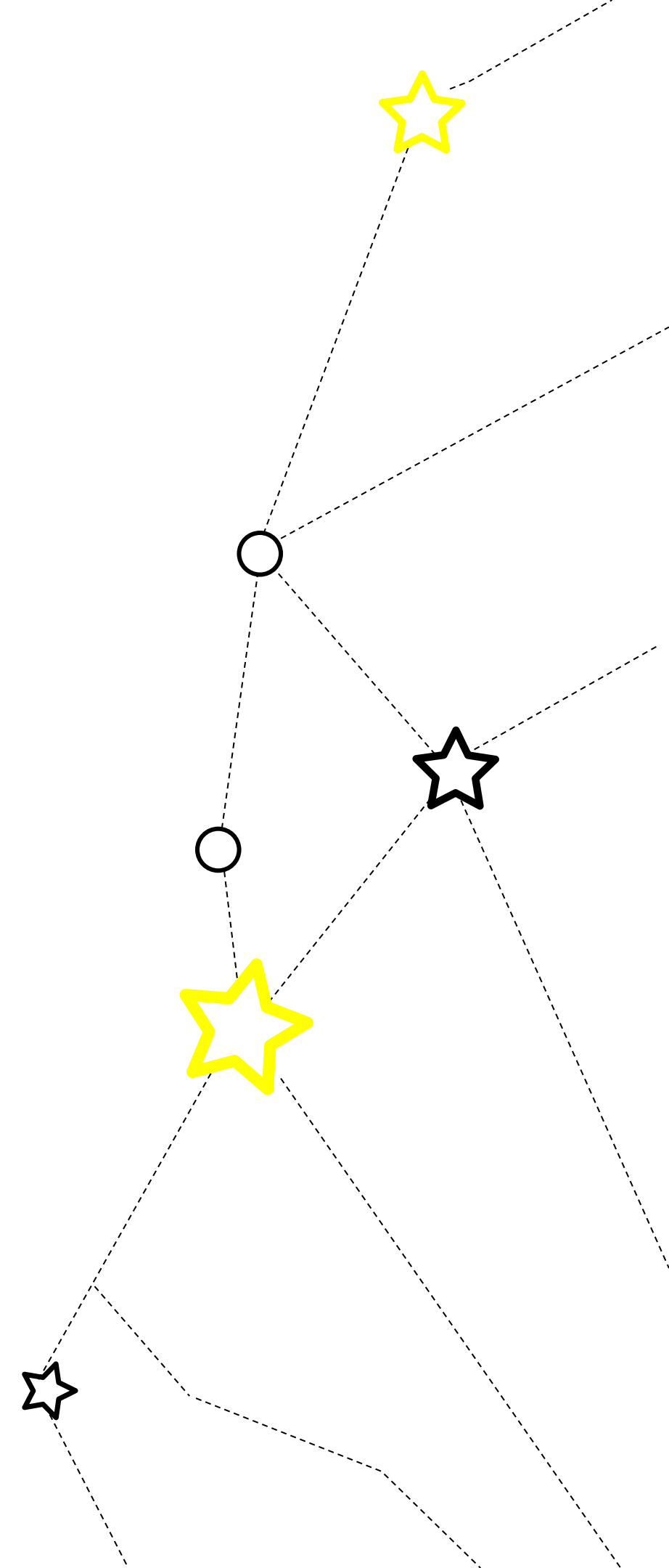
Собака: имеет дополнительный атрибут под названием порода (строка) и метод под названием лай(), который печатает **"Гав!"**.



Кошка: имеет дополнительный атрибут под названием цвет (строка) и метод под названием мяу(), который печатает **"Мяу!"**.

Убедитесь, что атрибуты каждого класса инкапсулированы, сделав их приватными.

Наконец, создайте экземпляры классов **Dog** и **Cat** и вызовите их соответствующие методы, чтобы продемонстрировать наследование, инкапсуляцию и полиморфизм.



Домашнее задание

```
class Pet:
    def __init__(self, name, species, age, gender):
        self._name = name
        self._species = species
        self._age = age
        self._gender = gender

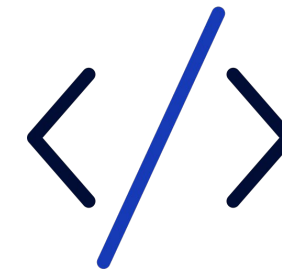
    def get_info(self):
        print(f'{self._name} ({self._gender}), {self._species}, {self._age} years old')

class Dog(Pet):
    def __init__(self, name, age, gender, breed):
        super().__init__(name, "Dog", age, gender)
        self._breed = breed

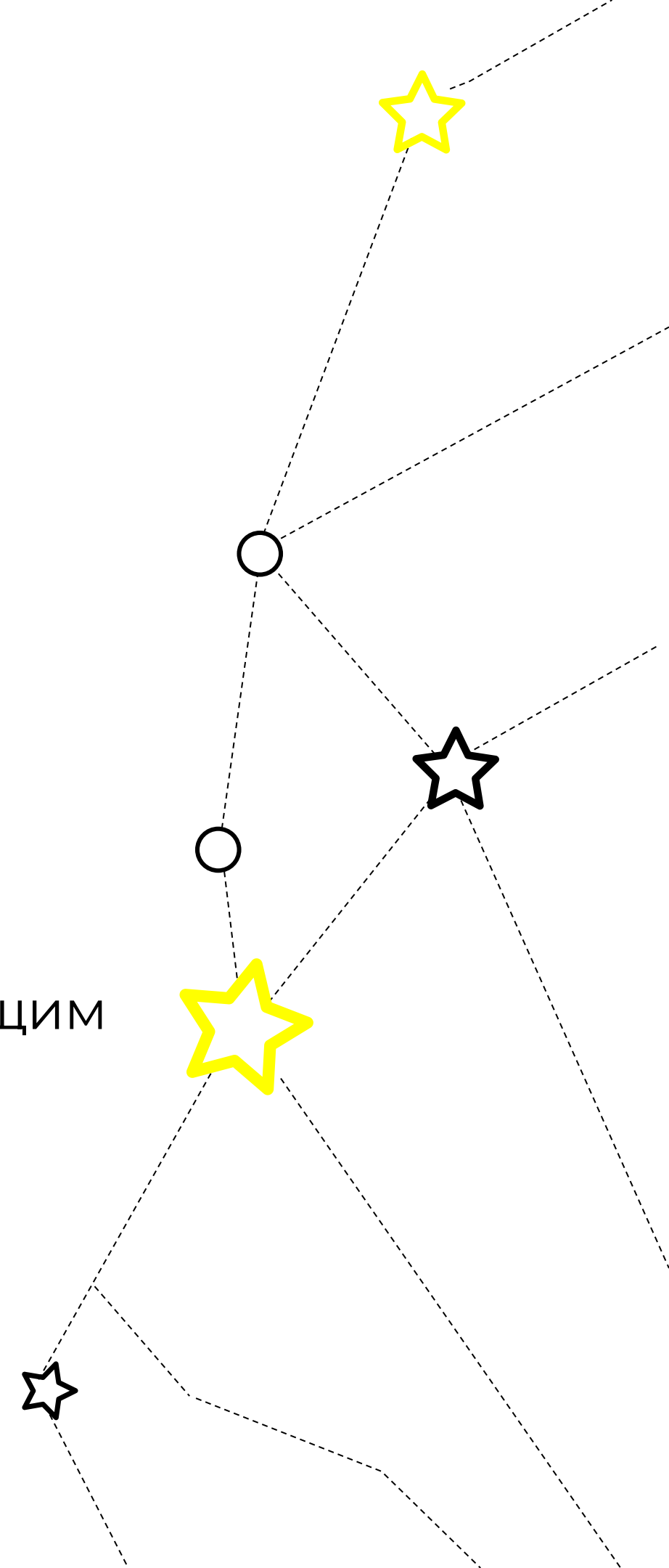
    def bark(self):
        print("Woof!")

class Cat(Pet):
    def __init__(self, name, age, gender, color):
        super().__init__(name, "Cat", age, gender)
        self._color = color

    def meow(self):
        print("Meow!")
```



Например,
ваш код может
выглядеть следующим
образом

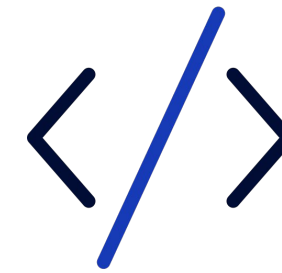


Домашнее задание

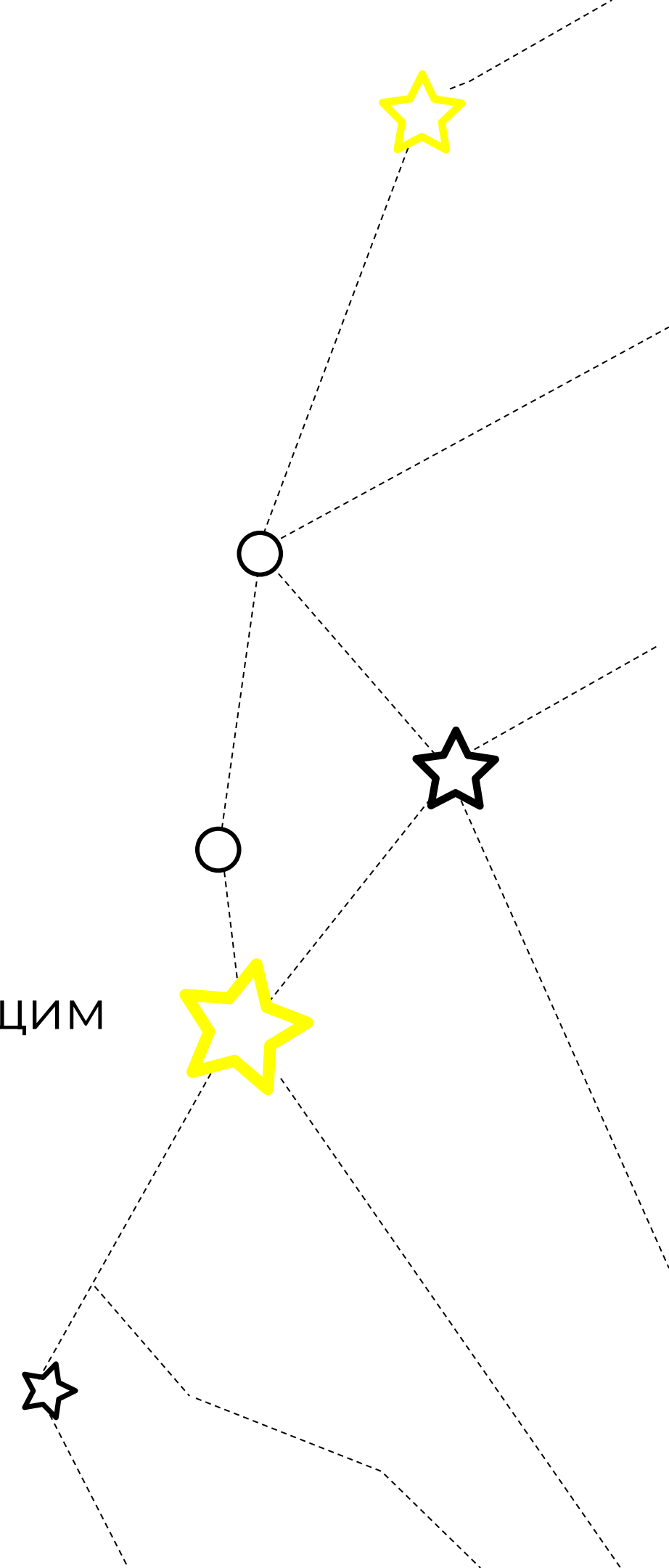
```
# create instances of the Dog and Cat classes
dog = Dog("Buddy", 2, "Male", "Golden Retriever")
cat = Cat("Fluffy", 3, "Female", "Calico")

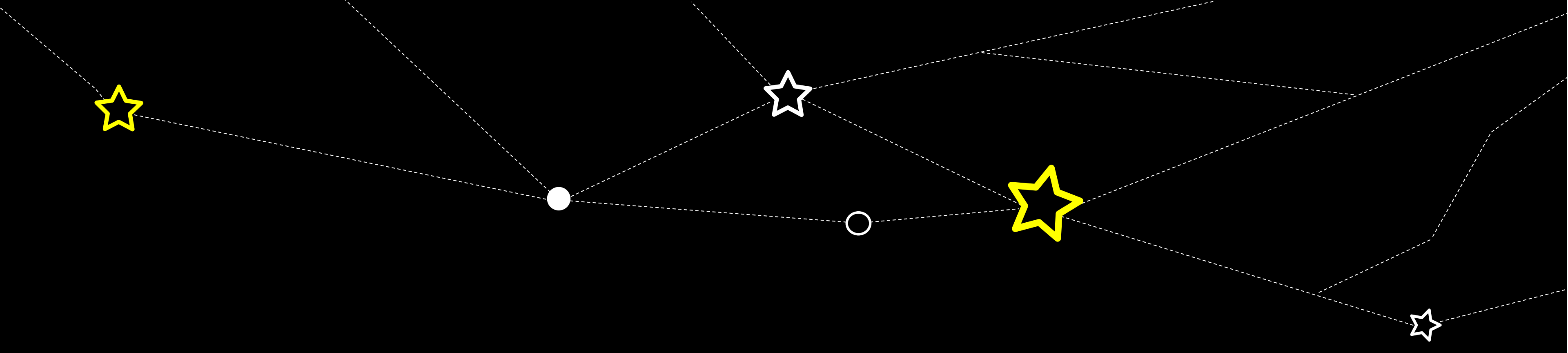
# call the methods on the objects to demonstrate
inheritance, encapsulation, and polymorphism
dog.get_info()
dog.bark()

cat.get_info()
cat.meow()
```



Например,
ваш код может
выглядеть следующим
образом





СПАСИБО ЗА ВНИМАНИЕ

Михаил Бородин



mikhailborodin.com